

基于 FreeRTOS 的智能家居环境监控系统 研究与实现

——从裸机到实时操作系统的完整移植与调试

作者姓名

2026 年 5 月 10 日

摘要

本文详细记录了一套基于 FreeRTOS 的智能家居环境监控系统的完整开发历程。系统以 ARM Cortex-M3 架构为核心，采用 QEMU 仿真平台进行验证，实现了温湿度数据采集、数字滤波、实时显示及高温报警等功能。论文重点阐述了从裸机程序到 FreeRTOS 多任务系统的移植过程，并深入分析了一个典型的实时操作系统启动故障：FreeRTOS 任务在 QEMU 环境中无法执行的问题。通过反汇编分析、QEMU 中断追踪和符号表检查等手段，最终定位到 `configASSERT` 宏在 NVIC 优先级位数检查上的断言失败，导致调度器陷入死循环。本研究为嵌入式实时系统在仿真环境中的调试提供了可复现的问题分析范式，对嵌入式教学与工程实践具有参考价值。

关键词：FreeRTOS；Cortex-M3；QEMU；嵌入式系统；实时操作系统；调试方法

目录

1 绪论	4
1.1 研究背景与意义	4
1.2 国内外研究现状	4
1.3 论文组织结构	4
2 相关技术基础	5
2.1 ARM Cortex-M3 处理器架构	5
2.2 FreeRTOS 实时操作系统	5
2.3 QEMU 仿真平台	5
2.4 ARM Semihosting 技术	6
3 系统设计与实现	6
3.1 需求分析	6
3.2 软件架构设计	6
3.3 关键算法实现	7
4 绪论	11
4.1 研究背景与意义	11
4.2 国内外研究现状	11
4.3 论文组织结构	11
5 相关技术基础	12
5.1 ARM Cortex-M3 处理器架构	12
5.2 FreeRTOS 实时操作系统	12
5.3 QEMU 仿真平台	12
5.4 ARM Semihosting 技术	13
6 系统设计与实现	13
6.1 需求分析	13
6.2 软件架构设计	13
6.3 关键算法实现	14
7 问题分析与调试过程	15
7.1 故障现象描述	15
7.2 初步假设与排查	15
7.3 极简测试法缩小范围	16
7.4 QEMU 中断追踪分析	16
7.5 反汇编定位死循环	17

7.6	根因确认与修复	18
7.7	调试方法论总结	18
8	实验验证与结果分析	19
8.1	实验环境	19
8.2	功能验证	19
8.3	结果分析	19
9	结论与展望	20
9.1	主要研究成果	20
9.2	后续工作展望	20
	参考文献	22
A	FreeRTOSConfig.h 核心配置	23
B	调试命令速查表	23

1 绪论

1.1 研究背景与意义

随着物联网技术的快速发展，智能家居系统已成为嵌入式技术的重要应用领域。环境监控作为智能家居的基础功能，需要实时采集温度、湿度等环境参数，并进行数据处理和异常报警。这类应用对系统的实时性、可靠性和可扩展性提出了较高要求。

传统的裸机开发方式（Bare-metal）采用前后台循环或中断驱动模型，代码耦合度高，难以应对复杂的多传感器协同场景。FreeRTOS 作为一款轻量级、开源的实时操作系统，提供了任务调度、队列通信、信号量同步等机制，能够有效解耦系统功能模块，提高代码的可维护性。

然而，FreeRTOS 的引入也带来了新的复杂性。开发者不仅需要理解实时调度原理，还需要正确配置中断优先级、堆栈大小、时钟节拍等参数。尤其在仿真环境（如 QEMU）中，硬件行为的模拟可能与真实芯片存在差异，导致难以复现的启动故障。因此，研究 FreeRTOS 在仿真环境中的移植与调试方法，具有重要的工程实践价值。

1.2 国内外研究现状

FreeRTOS 由 Richard Barry 于 2003 年发布，目前已广泛应用于工业控制、汽车电子、医疗设备等领域。其内核代码简洁（核心代码约 9000 行），支持 35 种以上处理器架构，是嵌入式领域最受欢迎的 RTOS 之一。

在仿真验证方面，QEMU（Quick Emulator）支持多种 ARM Cortex-M 系列处理器的模拟，包括 Cortex-M0/M3/M4/M7 等。开发者可以在没有硬件的情况下进行系统验证，显著降低开发成本。但在 QEMU 中运行 FreeRTOS 时，由于中断控制器（NVIC）的模拟行为与真实硬件存在差异，常常会出现启动异常、任务不调度等问题，相关技术文档较为分散。

1.3 论文组织结构

本文按照“问题提出 → 系统设计 → 故障排查 → 实验验证”的逻辑展开：

- 第 5 章介绍相关技术基础，包括 Cortex-M3 架构、FreeRTOS 内核机制、QEMU 仿真原理和 ARM Semihosting 技术；
- 第 6 章阐述系统需求分析、软件架构设计和关键算法实现；
- 第 7 章详细记录 FreeRTOS 任务不执行的完整调试过程，这是本文的核心创新点；
- 第 8 章展示实验结果，验证系统的功能正确性和稳定性；
- 第 9 章总结研究成果，并提出后续优化方向。

2 相关技术基础

2.1 ARM Cortex-M3 处理器架构

ARM Cortex-M3 是 ARMv7-M 架构的 32 位处理器，专为嵌入式实时应用设计。其核心特性包括：

1. **Thumb-2 指令集**：混合 16 位和 32 位指令，兼顾代码密度和执行效率；
2. **嵌套向量中断控制器 (NVIC)**：支持最多 240 个外部中断和 1 个不可屏蔽中断 (NMI)，具有 4 256 级可编程优先级；
3. **SysTick 定时器**：24 位递减计数器，专为操作系统节拍设计；
4. **双堆栈模型**：主堆栈指针 (MSP) 用于异常处理，进程堆栈指针 (PSP) 用于任务上下文。

Cortex-M3 的异常向量表位于存储器起始地址，包含初始栈顶地址和各类异常处理程序入口。FreeRTOS 的上下文切换依赖于 PendSV 异常和 SysTick 中断，因此向量表的正确配置是系统启动的前提条件。

2.2 FreeRTOS 实时操作系统

FreeRTOS 采用抢占式优先级调度算法，其核心组件包括：

(1) 任务管理

任务 (Task) 是 FreeRTOS 调度的基本单位。每个任务拥有独立的栈空间和任务控制块 (TCB)。xTaskCreate() 函数负责分配 TCB 和栈内存，并将任务插入就绪列表。vTaskStartScheduler() 启动调度器，创建 Idle 任务后，通过 SVC 0 指令触发第一个任务的执行。

(2) 队列通信

队列 (Queue) 是任务间数据传递的主要机制。xQueueCreate() 创建指定长度和元素大小的队列，xQueueSend() 和 xQueueReceive() 实现阻塞式数据传输。队列内部使用临界区保护，确保多任务环境下的数据一致性。

(3) 内存管理

FreeRTOS 提供 5 种堆管理方案 (heap_1 至 heap_5)。本系统采用 heap_4，支持内存碎片合并，适用于频繁创建和删除任务的场景。

2.3 QEMU 仿真平台

QEMU 是一款开源的机器模拟器和虚拟化工具。对于 ARM Cortex-M 系列，QEMU 提供了以下功能：

- 指令级模拟：逐条解释执行 ARM Thumb 指令；
- 外设模拟：包括 UART、GPIO、Timer 等基本外设；
- Semihosting 支持：通过 BKPT 0xAB 指令陷阱，将目标程序的输出重定向到主机终端；
- GDB Stub：支持远程调试，可设置断点和单步执行。

本研究使用 mps2-an385 机器模型，该模型基于 ARM Cortex-M3 的 MPS2 开发板，具有 4MB ZBT SSRAM 和完整的 NVIC 模拟。

2.4 ARM Semihosting 技术

Semihosting 是 ARM 定义的一种调试机制，允许目标程序调用主机上的 I/O 功能。在 Cortex-M 处理器中，Semihosting 通过执行 BKPT 0xAB 指令触发调试事件。QEMU 拦截该事件后，根据寄存器 R0（功能号）和 R1（参数指针）执行相应的宿主操作，如文件读写、控制台输出等。

本系统的调试输出完全依赖 Semihosting 实现，无需配置真实的 UART 外设，简化了仿真环境的搭建流程。

3 系统设计与实现

3.1 需求分析

本系统模拟一套智能家居环境监控设备，核心功能需求如表 2 所示。

表 1: 系统功能需求

编号	功能模块	描述
R1	数据采集	模拟温湿度传感器，每 1 秒产生一组数据
R2	数字滤波	对原始数据进行 5 点滑动平均滤波
R3	实时显示	通过串口输出格式化后的温湿度信息
R4	高温报警	温度超过 30.0°C 时触发报警提示
R5	多任务调度	各功能模块独立运行，互不阻塞

3.2 软件架构设计

系统采用分层架构设计，如图 ?? 所示的逻辑结构。

(1) 硬件抽象层

包括启动代码 (startup.c) 和链接脚本 (linker_script.ld)。启动代码负责初始化向量表、复制数据段、清零 BSS 段, 最终跳转到 main() 函数。链接脚本定义了 Flash (64KB, 起始地址 0x00000000) 和 RAM (20KB, 起始地址 0x20000000) 的分布。

(2) 操作系统层

FreeRTOS 内核提供任务调度、队列通信和内存管理。系统配置 8MHz CPU 时钟、1kHz tick 频率、10KB 堆空间。

(3) 应用层

包含两个核心任务:

- **Sensor 任务** (优先级 2, 栈 512 字): 生成正弦波叠加随机噪声的模拟数据, 通过队列发送给 Display 任务;
- **Display 任务** (优先级 1, 栈 512 字): 从队列接收数据, 执行 5 点滑动平均滤波, 格式化输出到控制台, 并判断是否需要报警。

3.3 关键算法实现

(1) 传感器数据模拟

温度数据采用正弦波模拟一天内的温度变化:

$$T(t) = 27.0 + 7.5 \times \sin\left(\frac{2\pi t}{300}\right) + \mathcal{N}(0, 1.5) \quad (1)$$

其中 t 为采样序号, $\mathcal{N}(0, 1.5)$ 表示标准差为 1.5 的高斯噪声。湿度数据采用类似的模型, 并附加与温度的负相关性。

(2) 滑动平均滤波

系统实现了固定窗口大小为 5 的滑动平均滤波器:

$$y[n] = \frac{1}{5} \sum_{i=0}^4 x[n-i] \quad (2)$$

为提高嵌入式平台的执行效率, 采用定点数运算: 将所有温度值乘以 10 转换为整数, 滤波完成后再除以 10 恢复精度。这种设计避免了浮点运算单元 (FPU) 的依赖, 适用于 Cortex-M3 等无硬件浮点的处理器。

(3) 队列通信协议

定义统一的数据结构 SensorData_t:

```
1 typedef struct {
2     int16_t temperature; /* 真实温度 * 10 */
3     int16_t humidity;    /* 真实湿度 * 10 */
4     uint32_t timestamp;  /* 采样时间戳 */
```


5

} SensorData_t;

Listing 1: 传感器数据结构定义

队列深度设为 5，元素大小为 sizeof(SensorData_t)。Sensor 任务每 1 秒调用 xQueueSend() 写入数据，Display 任务通过 xQueueReceive() 阻塞读取，实现生产-消费者同步模型。

[12pt,a4paper]ctexart
geometry left=2.5cm,right=2.5cm,top=2.5cm,bottom=2.5cm graphicx amsmath,amsfonts,amssymb
booktabs longtable array multirow listings xcolor hyperref enumitem fancyhdr setspace
caption subcaption float

基于 FreeRTOS 的智能家居环境监控系统
研究与实现
——从裸机到实时操作系统的完整移植与调试 作者姓名 2026 年 5 月 10 日

摘要

本文详细记录了一套基于 FreeRTOS 的智能家居环境监控系统的完整开发历程。系统以 ARM Cortex-M3 架构为核心，采用 QEMU 仿真平台进行验证，实现了温湿度数据采集、数字滤波、实时显示及高温报警等功能。论文重点阐述了从裸机程序到 FreeRTOS 多任务系统的移植过程，并深入分析了一个典型的实时操作系统启动故障：FreeRTOS 任务在 QEMU 环境中无法执行的问题。通过反汇编分析、QEMU 中断追踪和符号表检查等手段，最终定位到 `configASSERT` 宏在 NVIC 优先级位数检查上的断言失败，导致调度器陷入死循环。本研究为嵌入式实时系统在仿真环境中的调试提供了可复现的问题分析范式，对嵌入式教学与工程实践具有参考价值。

关键词：FreeRTOS；Cortex-M3；QEMU；嵌入式系统；实时操作系统；调试方法

目录

4 绪论

4.1 研究背景与意义

随着物联网技术的快速发展，智能家居系统已成为嵌入式技术的重要应用领域。环境监控作为智能家居的基础功能，需要实时采集温度、湿度等环境参数，并进行数据处理和异常报警。这类应用对系统的实时性、可靠性和可扩展性提出了较高要求。

传统的裸机开发方式（Bare-metal）采用前后台循环或中断驱动模型，代码耦合度高，难以应对复杂的多传感器协同场景。FreeRTOS 作为一款轻量级、开源的实时操作系统，提供了任务调度、队列通信、信号量同步等机制，能够有效解耦系统功能模块，提高代码的可维护性。

然而，FreeRTOS 的引入也带来了新的复杂性。开发者不仅需要理解实时调度原理，还需要正确配置中断优先级、堆栈大小、时钟节拍等参数。尤其在仿真环境（如 QEMU）中，硬件行为的模拟可能与真实芯片存在差异，导致难以复现的启动故障。因此，研究 FreeRTOS 在仿真环境中的移植与调试方法，具有重要的工程实践价值。

4.2 国内外研究现状

FreeRTOS 由 Richard Barry 于 2003 年发布，目前已广泛应用于工业控制、汽车电子、医疗设备等领域。其内核代码简洁（核心代码约 9000 行），支持 35 种以上处理器架构，是嵌入式领域最受欢迎的 RTOS 之一。

在仿真验证方面，QEMU（Quick Emulator）支持多种 ARM Cortex-M 系列处理器的模拟，包括 Cortex-M0/M3/M4/M7 等。开发者可以在没有硬件的情况下进行系统验证，显著降低开发成本。但在 QEMU 中运行 FreeRTOS 时，由于中断控制器（NVIC）的模拟行为与真实硬件存在差异，常常会出现启动异常、任务不调度等问题，相关技术文档较为分散。

4.3 论文组织结构

本文按照“问题提出 → 系统设计 → 故障排查 → 实验验证”的逻辑展开：

- 第 5 章介绍相关技术基础，包括 Cortex-M3 架构、FreeRTOS 内核机制、QEMU 仿真原理和 ARM Semihosting 技术；
- 第 6 章阐述系统需求分析、软件架构设计和关键算法实现；
- 第 7 章详细记录 FreeRTOS 任务不执行的完整调试过程，这是本文的核心创新点；
- 第 8 章展示实验结果，验证系统的功能正确性和稳定性；
- 第 9 章总结研究成果，并提出后续优化方向。

5 相关技术基础

5.1 ARM Cortex-M3 处理器架构

ARM Cortex-M3 是 ARMv7-M 架构的 32 位处理器，专为嵌入式实时应用设计。其核心特性包括：

1. **Thumb-2 指令集**：混合 16 位和 32 位指令，兼顾代码密度和执行效率；
2. **嵌套向量中断控制器 (NVIC)**：支持最多 240 个外部中断和 1 个不可屏蔽中断 (NMI)，具有 4 256 级可编程优先级；
3. **SysTick 定时器**：24 位递减计数器，专为操作系统节拍设计；
4. **双堆栈模型**：主堆栈指针 (MSP) 用于异常处理，进程堆栈指针 (PSP) 用于任务上下文。

Cortex-M3 的异常向量表位于存储器起始地址，包含初始栈顶地址和各类异常处理程序入口。FreeRTOS 的上下文切换依赖于 PendSV 异常和 SysTick 中断，因此向量表的正确配置是系统启动的前提条件。

5.2 FreeRTOS 实时操作系统

FreeRTOS 采用抢占式优先级调度算法，其核心组件包括：

(1) 任务管理

任务 (Task) 是 FreeRTOS 调度的基本单位。每个任务拥有独立的栈空间和任务控制块 (TCB)。xTaskCreate() 函数负责分配 TCB 和栈内存，并将任务插入就绪列表。vTaskStartScheduler() 启动调度器，创建 Idle 任务后，通过 SVC 0 指令触发第一个任务的执行。

(2) 队列通信

队列 (Queue) 是任务间数据传递的主要机制。xQueueCreate() 创建指定长度和元素大小的队列，xQueueSend() 和 xQueueReceive() 实现阻塞式数据传输。队列内部使用临界区保护，确保多任务环境下的数据一致性。

(3) 内存管理

FreeRTOS 提供 5 种堆管理方案 (heap_1 至 heap_5)。本系统采用 heap_4，支持内存碎片合并，适用于频繁创建和删除任务的场景。

5.3 QEMU 仿真平台

QEMU 是一款开源的机器模拟器和虚拟化工具。对于 ARM Cortex-M 系列，QEMU 提供了以下功能：

- 指令级模拟：逐条解释执行 ARM Thumb 指令；
- 外设模拟：包括 UART、GPIO、Timer 等基本外设；
- Semihosting 支持：通过 BKPT 0xAB 指令陷阱，将目标程序的输出重定向到主机终端；
- GDB Stub：支持远程调试，可设置断点和单步执行。

本研究使用 mps2-an385 机器模型，该模型基于 ARM Cortex-M3 的 MPS2 开发板，具有 4MB ZBT SSRAM 和完整的 NVIC 模拟。

5.4 ARM Semihosting 技术

Semihosting 是 ARM 定义的一种调试机制，允许目标程序调用主机上的 I/O 功能。在 Cortex-M 处理器中，Semihosting 通过执行 BKPT 0xAB 指令触发调试事件。QEMU 拦截该事件后，根据寄存器 R0（功能号）和 R1（参数指针）执行相应的宿主操作，如文件读写、控制台输出等。

本系统的调试输出完全依赖 Semihosting 实现，无需配置真实的 UART 外设，简化了仿真环境的搭建流程。

6 系统设计与实现

6.1 需求分析

本系统模拟一套智能家居环境监控设备，核心功能需求如表 2 所示。

表 2: 系统功能需求

编号	功能模块	描述
R1	数据采集	模拟温湿度传感器，每 1 秒产生一组数据
R2	数字滤波	对原始数据进行 5 点滑动平均滤波
R3	实时显示	通过串口输出格式化后的温湿度信息
R4	高温报警	温度超过 30.0°C 时触发报警提示
R5	多任务调度	各功能模块独立运行，互不阻塞

6.2 软件架构设计

系统采用分层架构设计，如图 ?? 所示的逻辑结构。

(1) 硬件抽象层

包括启动代码 (startup.c) 和链接脚本 (linker_script.ld)。启动代码负责初始化向量表、复制数据段、清零 BSS 段, 最终跳转到 main() 函数。链接脚本定义了 Flash (64KB, 起始地址 0x00000000) 和 RAM (20KB, 起始地址 0x20000000) 的分布。

(2) 操作系统层

FreeRTOS 内核提供任务调度、队列通信和内存管理。系统配置 8MHz CPU 时钟、1kHz tick 频率、10KB 堆空间。

(3) 应用层

包含两个核心任务:

- **Sensor 任务** (优先级 2, 栈 512 字): 生成正弦波叠加随机噪声的模拟数据, 通过队列发送给 Display 任务;
- **Display 任务** (优先级 1, 栈 512 字): 从队列接收数据, 执行 5 点滑动平均滤波, 格式化输出到控制台, 并判断是否需要报警。

6.3 关键算法实现

(1) 传感器数据模拟

温度数据采用正弦波模拟一天内的温度变化:

$$T(t) = 27.0 + 7.5 \times \sin\left(\frac{2\pi t}{300}\right) + \mathcal{N}(0, 1.5) \quad (3)$$

其中 t 为采样序号, $\mathcal{N}(0, 1.5)$ 表示标准差为 1.5 的高斯噪声。湿度数据采用类似的模型, 并附加与温度的负相关性。

(2) 滑动平均滤波

系统实现了固定窗口大小为 5 的滑动平均滤波器:

$$y[n] = \frac{1}{5} \sum_{i=0}^4 x[n-i] \quad (4)$$

为提高嵌入式平台的执行效率, 采用定点数运算: 将所有温度值乘以 10 转换为整数, 滤波完成后再除以 10 恢复精度。这种设计避免了浮点运算单元 (FPU) 的依赖, 适用于 Cortex-M3 等无硬件浮点的处理器。

(3) 队列通信协议

定义统一的数据结构 SensorData_t:

```
1 typedef struct {
2     int16_t temperature; /* 真实温度 * 10 */
3     int16_t humidity;    /* 真实湿度 * 10 */
4     uint32_t timestamp;  /* 采样时间戳 */
```

```
5 } SensorData_t;
```

Listing 2: 传感器数据结构定义

队列深度设为 5，元素大小为 `sizeof(SensorData_t)`。Sensor 任务每 1 秒调用 `xQueueSend()` 写入数据，Display 任务通过 `xQueueReceive()` 阻塞读取，实现生产-消费者同步模型。

7 问题分析与调试过程

7.1 故障现象描述

在裸机版本 (Bare-metal) 验证通过后，开始移植 FreeRTOS 多任务版本。编译和链接均成功 (ELF 文件约 26KB，BIN 文件约 14KB)，但在 QEMU 中运行时出现如下异常现象：

1. 程序启动后仅打印启动横幅 (Banner)，随后 QEMU 超时终止；
2. 没有任何任务输出信息；
3. 无 HardFault、BusFault 等异常提示；
4. 程序行为表现为“静默死锁”，而非崩溃。

这一症状具有极强的迷惑性：编译器未报错、链接器未报错、程序能够执行到 `main()` 函数，但 FreeRTOS 任务似乎“消失”了。

7.2 初步假设与排查

基于现象，提出以下假设并逐一验证：

假设 1：栈空间不足导致任务创建失败

FreeRTOS 任务创建时需要从堆中分配栈内存。如果 `configTOTAL_HEAP_SIZE` 或任务栈大小设置过小，`xTaskCreate()` 可能失败。验证方法：将堆大小从 8KB 逐步增加到 10KB，任务栈统一设为 512 字。结果：症状未改善。

假设 2：软件定时器初始化异常

FreeRTOS 在启用软件定时器 (`configUSE_TIMERS=1`) 时会自动创建 Timer Service 任务。如果该任务初始化失败，可能导致调度器异常。验证方法：将 `configUSE_TIMERS` 设为 0，禁用所有定时器功能。结果：症状未改善。

假设 3：SysTick 配置错误

FreeRTOS 的时钟节拍依赖 SysTick 定时器。如果 `configCPU_CLOCK_HZ` 或 `configTICK_RATE_HZ` 配置错误，可能导致时钟节拍不触发。验证方法：检查配置值为 8MHz / 1kHz，计算重载值 $8000000/1000=8000$ ，在 24 位计数器范围内。结果：配置正确，症状未改善。

假设 4：中断向量表配置错误

FreeRTOS 依赖 SVC、PendSV 和 SysTick 三个异常处理程序。如果向量表未正确映射 FreeRTOS 的处理函数，可能导致上下文切换失败。验证方法：检查符号表，确认 SVC_Handler 为强符号 (Strong Symbol)，地址为 0x00001070，且反汇编代码符合 vPortSVCHandler 的实现特征（恢复任务上下文、设置 PSP、返回 Thread 模式）。结果：中断向量表配置正确。

经过以上排查，四个常见假设均被排除，故障原因更加扑朔迷离。

7.3 极简测试法缩小范围

为区分“任务未启动”和“Semihosting 在任务中失效”两种可能性，编写极简测试程序：

```

1 static void vTaskTest(void *pvParameters) {
2     (void)pvParameters;
3     sh_write0("INSIDE TASK\r\n");
4     for (;;) { __asm volatile("nop"); }
5 }
6
7 int main(void) {
8     sh_write0("BEFORE SCHEDULER\r\n");
9     xTaskCreate(vTaskTest, "Test", 512, NULL, 1, NULL);
10    sh_write0("AFTER CREATE\r\n");
11    vTaskStartScheduler();
12    sh_write0("SCHEDULER RETURNED\r\n");
13    for (;;) ;
14 }

```

Listing 3: 极简任务测试程序

运行结果：

```

BEFORE SCHEDULER
AFTER CREATE

```

vTaskStartScheduler() 未返回（符合预期），但 INSIDE TASK 未出现。这一结果排除了“Semihosting 失效”的假设，确认问题根源在于 **任务根本没有被启动**。

7.4 QEMU 中断追踪分析

启用 QEMU 的 -d int 调试选项，追踪异常触发情况：

```
Loaded reset SP 0x20005000 PC 0x41 from vector table
Taking exception 16 [Semihosting call] on CPU 0
...handling as semihosting call 0x4
BEFORE SCHEDULER
Taking exception 16 [Semihosting call] on CPU 0
...handling as semihosting call 0x4
AFTER CREATE
qemu-system-arm: terminating on signal 15 from pid 1614 (timeout)
```

关键发现:没有任何 SVC 异常(Exception 11)被触发。正常情况下,vTaskStartScheduler()最终会调用 prvPortStartFirstTask(),后者执行 svc 0 指令触发 SVC 异常以启动第一个任务。SVC 异常的缺失意味着:

1. prvPortStartFirstTask() 未被执行; 或
2. xPortStartScheduler() 在调用 prvPortStartFirstTask() 之前已陷入死循环。

7.5 反汇编定位死循环

对 xPortStartScheduler() 进行反汇编分析,发现如下关键代码段:

```
000010bc <xPortStartScheduler>:
...
111a:    cmp     r3, #4
111c:    beq.n   1136 <xPortStartScheduler+0x7a>
111e:    mov.w   r3, #80
1122:    msr     BASEPRI, r3
1126:    isb     sy
112a:    dsb     sy
112e:    str     r3, [r7, #8]
1130:    nop
1132:    nop
1134:    b.n     1132 <xPortStartScheduler+0x76>
```

在地址 0x1134 处,指令 b.n 1132 构成了一个 **无限循环**。回溯该条件分支的上游逻辑:

1. 0x10c2-0x10da:向 NVIC 优先级寄存器写入 0xFF 并读回,得到 ucMaxPriorityValue;
2. 0x10dc-0x1110: 通过移位操作计算 ucMaxPriorityValue 的最高有效位位置,计数器初始值为 7;

3. 0x1116: rsb r3, r3, #7 计算优先级位数, 即 $r3 = 7 - \text{count}$;
4. 0x111a-0x111c: 比较 r3 是否等于 4, 若不相等则跳转至 0x1132 进入死循环。

这段代码对应 FreeRTOS 的 configASSERT 检查逻辑。其数学原理为: Cortex-M3 的 NVIC 优先级寄存器通常为 4 位有效 (即写入 0xFF 后读回 0xF0, 最高有效位在第 4 位)。FreeRTOS 断言读回值的最高有效位必须位于第 4 位 (从 0 开始计数), 即 $7 - \text{count} == 4$, 因此 $\text{count} == 3$ 。

然而, QEMU 的 mps2-an385 模型模拟的 NVIC 优先级寄存器读回值为 0xE0 (3 位有效), 导致 $\text{count} == 4$, $r3 = 7 - 4 = 3$, $r3 \neq 4$, 断言失败。

7.6 根因确认与修复

最终确认: QEMU mps2-an385 的 NVIC 优先级模拟为 3 位, 而 FreeRTOS 的 configASSERT 硬编码要求 4 位, 导致断言失败后进入无限循环。

修复方案: 在 FreeRTOSConfig.h 中注释掉 configASSERT 宏定义:

```
1 /* configASSERT disabled for QEMU mps2-an385
2    (NVIC priority bits mismatch) */
3 /* #define configASSERT( x ) if( ( x ) == 0 ) {
4    taskDISABLE_INTERRUPTS(); for( ;; ); } */
```

Listing 4: configASSERT 修复方案

需要说明的是, 禁用 configASSERT 仅适用于 QEMU 仿真环境。在真实 STM32F103 (Cortex-M3) 硬件上, NVIC 为标准的 4 位优先级, 可以重新启用断言以获取运行时保护。

修复后重新编译运行, 极简测试程序成功输出 INSIDE TASK, 确认任务已正常启动。

7.7 调试方法论总结

本次调试过程遵循了系统化的嵌入式故障排查方法论, 如表 3 所示。

表 3: 调试方法总结

阶段	方法	工具	结论
1	假设验证法	修改配置参数	排除栈空间、定时器、SysTick 问题
2	极简测试法	编写最小可复现程序	确认任务未启动, 非 Semihosting 问题
3	中断追踪法	QEMU -d int	发现 SVC 异常缺失
4	反汇编分析法	objdump -d	定位 xPortStartScheduler 死循环
5	代码关联法	对照 FreeRTOS 源码	确认 configASSERT 断言失败

8 实验验证与结果分析

8.1 实验环境

实验环境配置如表 4 所示。

表 4: 实验环境配置

项目	配置
宿主机操作系统	Windows 11 + WSL2 (Ubuntu 24.04)
交叉编译器	ARM GCC 13.2.1 (arm-none-eabi-gcc)
构建工具	GNU Make 4.3
仿真器	QEMU 8.2.2 (qemu-system-arm)
目标板模型	mps2-an385 (Cortex-M3)
实时操作系统	FreeRTOS V10.4.6

8.2 功能验证

运行 `./run_qemu.sh` 脚本，系统输出如下：

```
=====
Smart Home Environment Monitor
FreeRTOS + QEMU Simulation
=====

[1] Temp: 20.0 C | Humidity: 50.1 %
[2] Temp: 20.1 C | Humidity: 57.5 %
...
[36] Temp: 29.9 C | Humidity: 58.1 %
    >>> [ALARM #1] Temperature exceeded 30.0 C! <<<
[37] Temp: 30.2 C | Humidity: 56.7 %
    >>> [ALARM #2] Temperature exceeded 30.0 C! <<<
...
```

8.3 结果分析

(1) 任务调度验证

连续运行 156 个采样周期（约 156 秒），Sensor 任务和 Display 任务交替执行，无数据丢失、无死锁、无堆栈溢出。`vTaskDelay()` 精确实现 1 秒周期，验证了 SysTick 中断配置的正确性。

(2) 滤波效果验证

对比原始数据和滤波后数据，5 点滑动平均滤波器有效抑制了随机噪声，温度曲线平滑连续，未出现跳变或毛刺。

(3) 报警功能验证

温度超过 30.0°C 时，报警提示正确触发，报警计数器连续递增。当温度回落至 30°C 以下时，报警自动停止，hysteresis 行为符合预期。

(4) 资源占用分析

通过 arm-none-eabi-size 工具分析 ELF 文件：

表 5: 内存占用分析

段	大小 (字节)	占比	说明
.text	10,369	16.2%	代码段
.data	8	<0.1%	已初始化全局变量
.bss	12,592	19.7%	未初始化全局变量 (含 10KB 堆)
剩余 RAM	≈7,000	10.9%	可用栈空间

系统总内存占用约 23KB，远低于 64KB Flash 和 20KB RAM 的硬件限制，具有良好的可扩展性。

9 结论与展望

9.1 主要研究成果

本文完成了一套基于 FreeRTOS 的智能家居环境监控系统的完整设计与实现，主要贡献包括：

- 系统实现：**完成了从裸机到 FreeRTOS 多任务系统的移植，实现了温湿度采集、数字滤波、实时显示和高温报警四大功能模块；
- 问题定位：**发现并深入分析了一个 QEMU 仿真环境中的 FreeRTOS 启动故障，根因为 QEMU mps2-an385 的 NVIC 优先级位数（3 位）与 FreeRTOS 断言预期（4 位）不匹配；
- 方法论：**总结了一套适用于嵌入式 RTOS 调试的”假设验证 → 极简测试 → 中断追踪 → 反汇编分析 → 源码对照”五步法。

9.2 后续工作展望

未来可在以下方向继续深入研究：

1. **真实硬件迁移**：将系统迁移至 STM32F103 开发板，接入 DHT11/DS18B20 真实传感器，验证仿真到硬件的一致性；
2. **低功耗优化**：在空闲任务中实现 Sleep Mode，降低系统功耗，延长电池供电设备的续航时间；
3. **网络互联**：增加 ESP8266 WiFi 模块驱动，实现环境数据上传至 MQTT 云平台；
4. **协议栈扩展**：引入 lwIP 协议栈，支持 HTTP/WebSocket 远程监控界面。

参考文献

- [1] Barry R. Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide[M]. Real Time Engineers Ltd, 2016.
- [2] ARM Limited. ARM Cortex-M3 Technical Reference Manual[R]. ARM DDI 0337G, 2010.
- [3] ARM Limited. ARMv7-M Architecture Reference Manual[R]. ARM DDI 0403E, 2014.
- [4] QEMU Developers. QEMU System Emulation User's Manual[EB/OL]. <https://qemu.readthedocs.io>, 2024.
- [5] GNU Project. Using the GNU Compiler Collection (GCC)[EB/OL]. <https://gcc.gnu.org/onlinedocs>, 2024.
- [6] 刘火良, 杨森. STM32 库开发实战指南 [M]. 北京: 机械工业出版社, 2019.
- [7] Joseph Yiu. The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors[M]. 3rd ed. Newnes, 2013.
- [8] FreeRTOS.org. FreeRTOS Configurations[EB/OL]. <https://www.freertos.org/a00110.html>, 2024.
- [9] GitHub, Inc. SmartHomeEnvironmentMonitor Repository[EB/OL]. <https://github.com/chenby198>, 2024.

A FreeRTOSConfig.h 核心配置

```

1  #ifndef FREERTOS_CONFIG_H
2  #define FREERTOS_CONFIG_H
3
4  #define configUSE_PREEMPTION          1
5  #define configCPU_CLOCK_HZ           8000000
6  #define configTICK_RATE_HZ           1000
7  #define configMAX_PRIORITIES          5
8  #define configMINIMAL_STACK_SIZE      256
9  #define configTOTAL_HEAP_SIZE         10240
10 #define configCHECK_FOR_STACK_OVERFLOW 0
11 #define configUSE_TIMERS                0
12
13 /* Assert definition (disabled for QEMU mps2-an385) */
14 /* #define configASSERT( x ) if( ( x ) == 0 ) {
15     taskDISABLE_INTERRUPTS(); for( ;; ); } */
16
17 #define vPortSVCHandler                SVC_Handler
18 #define xPortPendSVHandler              PendSV_Handler
19 #define xPortSysTickHandler             SysTick_Handler
20
21 #endif

```

Listing 5: FreeRTOSConfig.h 核心配置项

B 调试命令速查表

表 6: 常用调试命令

命令	功能
make clean && make	完整重新编译
arm-none-eabi-size *.elf	查看内存段大小
arm-none-eabi-nm *.elf	查看符号表
arm-none-eabi-objdump -d *.elf	反汇编代码
qemu-system-arm ... -d int	追踪中断异常
qemu-system-arm ... -s -S	启用 GDB 调试服务器